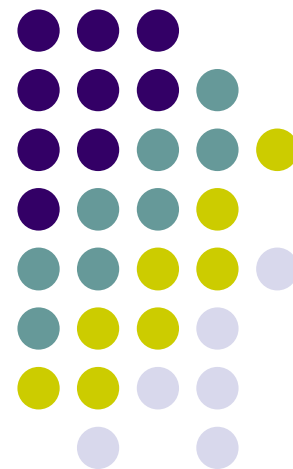


# 动规之背包

- 0/1背包
- 0/1背包降维
- 完全背包
- 多重背包
- 混合背包
- 二维费用背包
- 分组背包
- 有依赖的背包（提高组2006金明的预算）
- 背包的方案总数  $\max \rightarrow \text{sum}$



# 0/1背包



- [问题描述]

有一个吝啬的地主，不愿意给工人付工钱，就从家里的物品中取出 $N$ 个，对工人说，你可以从这些物品中任意挑选，只要你的口袋装得下。

现在已知物品的总数量 $N$ ，和工人的口袋大小 $M$ ，及每种物品所占的空间 $w_i$ ，物品的价值 $v_i$ 。求出该背包能装载的最大价值。（物品不可以分，不考虑物品间的缝隙）

$N=5$   $M=10$

| 空间 $w_i$ | $v_i$ 价值 |
|----------|----------|
|----------|----------|

|   |   |
|---|---|
| 2 | 8 |
|---|---|

|   |   |
|---|---|
| 8 | 6 |
|---|---|

|   |   |
|---|---|
| 3 | 4 |
|---|---|

|   |   |
|---|---|
| 5 | 4 |
|---|---|

|   |   |
|---|---|
| 3 | 3 |
|---|---|

输出：16

贪心：先选单位空间能获得价值更大者

2:8

3:4

3:3

15

正确选择：

2 8

3 4

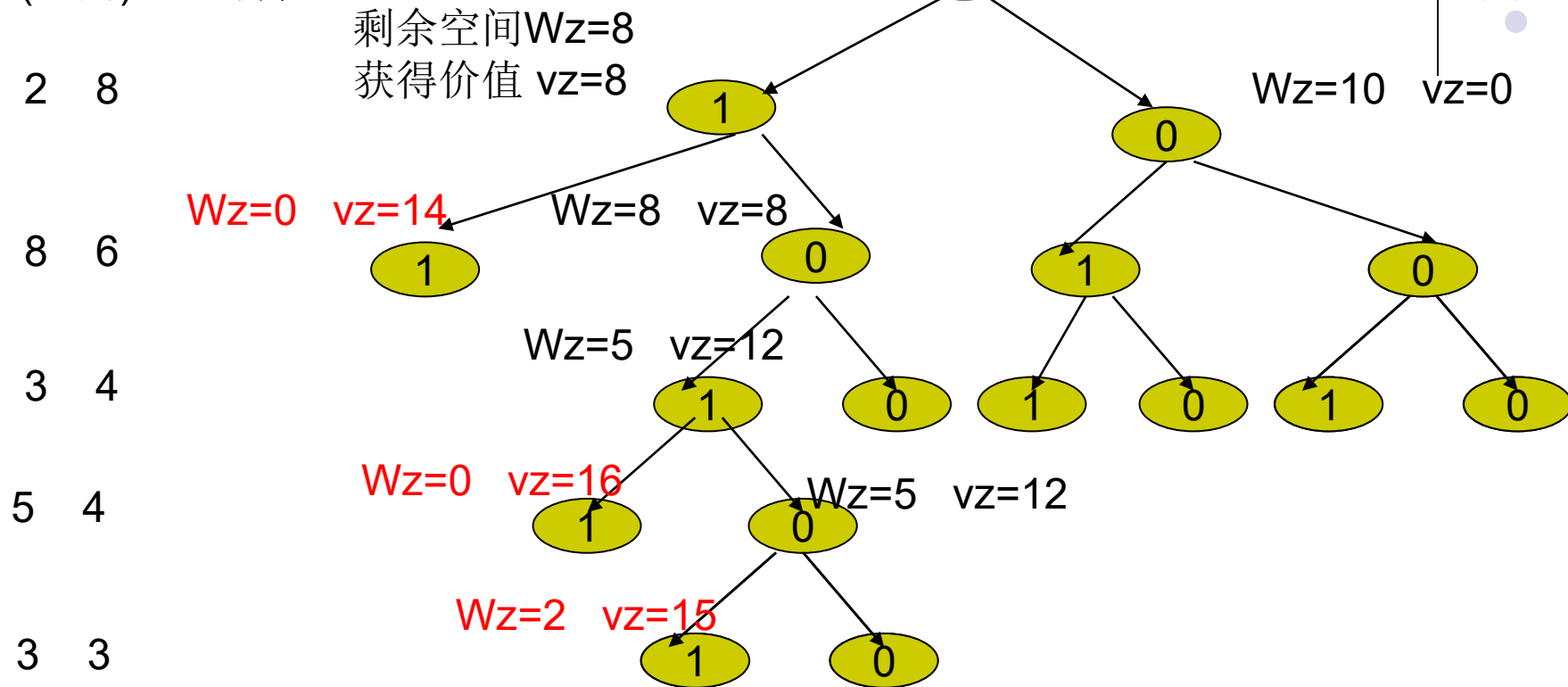
5 4

16

# 深度优先搜索：1表示选，0表示不选



Wi(空间) vi(价值)



当剩余空间为0时，不再递归，  
当5个物品都被选完 $dep=5$ 时，不再递归，

产生的总价值 $vz$ ，刷新 $maxt$

N=5 M=10

Wi Vi

2 8

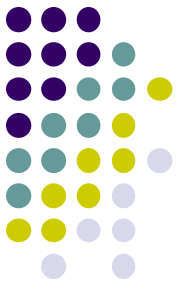
8 6

3 4

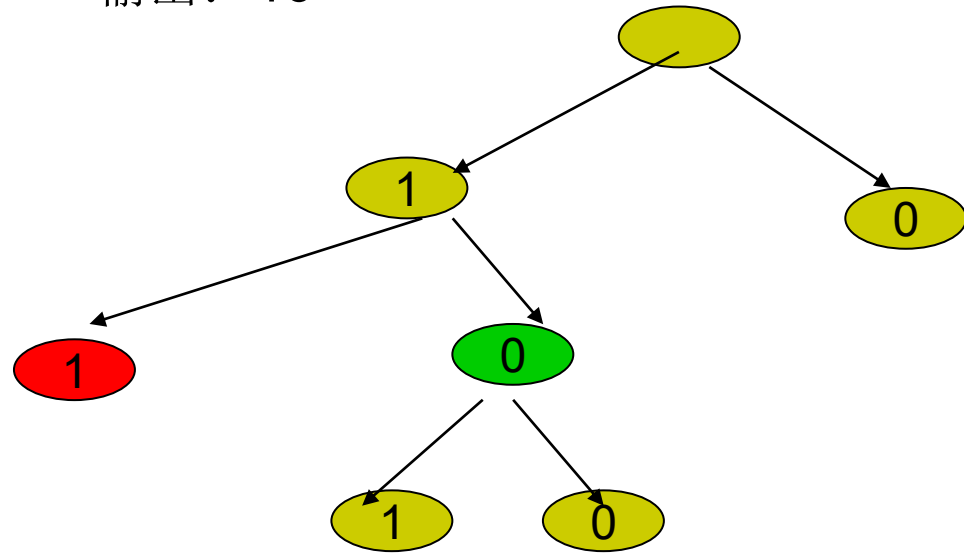
5 4

3 3

输出: 16



```
void dfs(int dep);
{int i;
  for(i=1;i>=0;i--)//选1方便调试
  if (wz>=w[dep]) //剩余空间能放该物品
  { wz=wz-w[dep]*i; //空间减少
    vz=vz+v[dep]*i; //价值增加
    if (dep==n) or (wz==0)
    { if (vz>max) max=vz; }
      else dsf(dep+1);
    wz=wz+w[dep]*i;
    vz=vz-v[dep]*i;
  }
}
```





## ● [问题描述]

有一个吝啬的地主，不愿意给工人付工钱，就从家里的物品中取出 $N$ 个，对工人说，你可以从这些物品中任意挑选，只要你的口袋装得下。

现在已知物品的总数量 $N$ ，和工人的口袋大小 $M$ ，及每种物品所占的空间 $w_i$ ，物品的价值 $v_i$ 。求出该背包能装载的最大价值。（物品不可以分，不考虑物品间的缝隙）

$N=5$   $M=10$

| 空间 $w_i$ | $v_i$ 价值 |
|----------|----------|
|----------|----------|

|   |   |
|---|---|
| 2 | 8 |
|---|---|

|   |   |
|---|---|
| 8 | 6 |
|---|---|

|   |   |
|---|---|
| 3 | 4 |
|---|---|

|   |   |
|---|---|
| 5 | 4 |
|---|---|

|   |   |
|---|---|
| 3 | 3 |
|---|---|

输出：16

动态规划解题步骤：

1. 状态设想，抽象子问题

$f[]$  一维？ 代表什么？

$f[][]$  二维？ 代表什么？

2. 初步规划动规方程

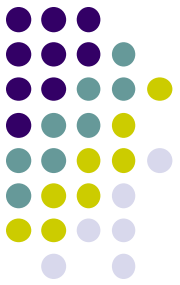
从某个中间状态思考来源

$F[] = \dots\dots\dots$   $F[][] = \dots\dots\dots$

3. 打表验证或找出正确的动规方程

4. 处理边界值

5. 代码实现



## 第一步：状态设想

总问题：N个物品，占用M个空间时所能获得的最大价值。

子问题： $f[i,j]$ ：前*i*个物品占用*j*个空间时能获得的最大价值。

## 第二步：初步规划动规方程

从某个中间状态思考来源

$F[i][j]=\dots\dots\dots$  设前面的任何决策都有答案了，当前决策？

|        |        | 0   | 1  | 2    | 3 | 4 | $j$ |   |   |   |   | $M$ |
|--------|--------|-----|----|------|---|---|-----|---|---|---|---|-----|
| $W[i]$ | $v[i]$ | 0   | 0  | 0    | 0 | 0 | 0   | 0 | 0 | 0 | 0 | 0   |
|        |        | 2   | 5  | 1    | 0 |   |     |   |   |   |   |     |
|        |        | 4   | 20 | 2    | 0 |   |     |   |   |   |   |     |
|        |        | 2   | 10 | $3i$ | 0 |   |     |   |   |   |   |     |
|        |        | $N$ | 0  |      |   |   |     |   |   |   |   |     |

选与不选  
有时选了总价值反而更小

要选当前

不选当前物品，4个空间全都给前两个物品来使用

+10

1:  $F[i][j]=F[i-1][j]$

2:  $F[i][j]=F[i-1][j]-W[i]+V[i]$

子问题:  $f[i,j]$ : 前*i*个物品占用*j*个空间时能获得的最大价值。

第三步: 打表 验证、确定动规方程

●  $N=5$   $M=10$

| $w_i$ 空 | $v_i$ 价 | J:0 | 1 | 2 | 3 | 4 | 5  | 6  | 7  | 8  | 9  | 10  |
|---------|---------|-----|---|---|---|---|----|----|----|----|----|-----|
| 2       | 8       | 0   | 0 | 8 | 8 | 8 | 8  | 8  | 8  | 8  | 8  | 8   |
| 8       | 6       | 0   | 0 | 8 | 8 | 8 | 8  | 8  | 8  | 8  | 8  | 8?  |
| 3       | 4       | 0   | 0 | 8 | 8 | 8 | 8? | 8? | 8? | 8? | 8? | 8?  |
| 5       | 4       | 0   | 0 | 8 | 8 | 8 | 12 | 12 | 12 | 12 | 12 | 14? |
| 3       | 3       | 0   | 0 | 8 | 8 | 8 | 12 | 12 | 12 | 12 | 12 | 16? |

1.放当前物品空间不够,把所有的空间都让给前面的物品

if ( $j < w[i]$ )  $f[i][j] = f[i-1][j]$

2.放当前物品空间足够,但放了当前物品产生的价值更小,仍然选择空间留给前面

if ( $(j \geq w[i]) \&\&$ 放了更小)  $f[i][j] = f[i-1][j]$

3.放当前物品空间足够,且放了比不放产生的价值更大,选择放

if ( $(j \geq w[i]) \&\&f[i-1][j-w[i]]+v[i] > f[i-1][j]$ )  $f[i][j] = f[i-1][j-w[i]]+v[i]$ ;



$f[i][j] = f[i-1][j];$  //初值设为不选

if ( $(j \geq w[i]) \ \&\& \ f[i-1][j-w[i]] + v[i] > f[i-1][j]$ )

$f[i][j] = f[i-1][j-w[i]] + v[i];$  //如果能选，且选了更优时，刷新初值

|               |      | 0 | 1                                    | 2 | 3 | 4 | $j$ |   |   |   |   | $M$ |
|---------------|------|---|--------------------------------------|---|---|---|-----|---|---|---|---|-----|
| $W[i] \ v[i]$ | 0    | 0 | 0                                    | 0 | 0 | 0 | 0   | 0 | 0 | 0 | 0 | 0   |
|               | 2 5  | 1 |                                      |   |   |   |     |   |   |   |   |     |
|               | 4 20 | 2 |                                      |   |   |   |     |   |   |   |   |     |
|               | 2 10 | 3 |                                      |   |   |   |     |   |   |   |   |     |
|               | $i$  |   |                                      |   |   |   |     |   |   |   |   |     |
| $N$           |      | 0 | 1: $F[i][j] = F[i-1][j]$             |   |   |   |     |   |   |   |   |     |
|               |      | 0 | 2: $F[i][j] = F[i-1][j-W[i]] + V[i]$ |   |   |   |     |   |   |   |   |     |

Diagram illustrating the dynamic programming table for the knapsack problem. The table has rows for items (0 to 3) and columns for capacity (0 to 4). The values in the table represent the maximum value achievable for each item and capacity. The current item being considered is item 3 (weight 2, value 10). The current capacity is 4. The value in the cell (3, 4) is highlighted in yellow, indicating it is the current state. An arrow points to this cell with the label "+10". A text box below the table states: "不选当前物品，4个空间全都给前两个物品来使用" (Do not select the current item, use all 4 space for the previous two items). Another text box points to the cell (3, 3) with the label "要选当前" (Select the current item).

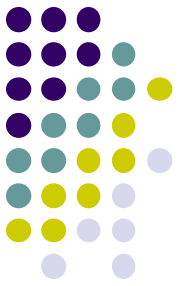
第四步：思考边界初值问题



```
#include<iostream>
#include<cmath>
using namespace std;
int f[31][30001],w[31],v[31];
```

运行时间:  $O(n*m)$

占用空间:  $n*m$



```
int main()
{
    int m,n,i,j;
    cin>>n>>m;
    for(i=1;i<=n;i++)
        cin>>w[i]>>v[i];

    for(i=1;i<=n;i++)
        for(j=1;j<=m;j++)
        {
            f[i][j]=f[i-1][j];
            if ((j>=w[i])&&f[i-1][j-w[i]]+v[i]>f[i-1][j])
                f[i][j]=f[i-1][j-w[i]]+v[i];
        }

    cout<<f[n][m];
    return 0;
}
```

代码可否这样优化: ?

```
For (int i=1;i<=n;i++)
    for (j=v[i];j<=m;j++)
    {f[i][j]=f[i-1][j];
      if ((j>=w[i])&&f[i-1][j-w[i]]+v[i]>f[i-1][j])
          f[i][j]=f[i-1][j-w[i]]+v[i];
    }
    每层的答案都要往下继承
```



例2：【单关键字背包问题】：有一个箱子容量为M(1~100)，同时有N(1~10)个物品，每个物品有一个体积，要求从这N个物品中任取若干个装入箱内，使得箱子的剩余空间最小,输出剩余空间。

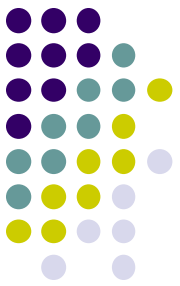
即：不超过总空间的情况下，占用的空间最大

输入：4 10 (n m)

6  
5  
3  
8  
输出： 1

解法一：转换为类似01背包的问题  
01背包：n个物品，不超过M空间的情况下，能占用的最大价值。  
本题： n个物品，不超过M空间的情况下，能占用的最大空间。  
将每个物品空间 当做 收集到的价值（空间价值）  
剩余的最小空间= $M-f[n][m]$ （总空间- 可能占用的最大空间价值）

|      |      | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|------|------|---|---|---|---|---|---|---|---|---|---|----|
| 占用空间 | 空间价值 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  |
| 6 6  | 1    | 0 | 0 | 0 | 0 | 0 | 0 | 6 | 6 | 6 | 6 | 6  |
| 5 5  | 2    | 0 | 0 | 0 | 0 | 0 | 0 | 6 | 6 | 6 | 6 | 6  |
| 3 3  | 3    | 0 | 0 | 0 | 3 | 3 | 3 | 6 | 6 | 6 | 9 | 9  |
| 8 8  | 4    | 0 | 0 | 0 | 3 | 3 | 3 | 6 | 6 | 8 | 9 | 9  |



【一本通，装箱问题】：有一个箱子容量为V(1~100)，同时有N(1~10)个物品，每个物品有一个体积，要求从这N个物品中任取若干个装入箱内，使得箱子的剩余空间最小,输出剩余空间。

输入：10 4

6  
5  
3  
8  
输出：1

辐射刷新法：播放ppt,分析过程

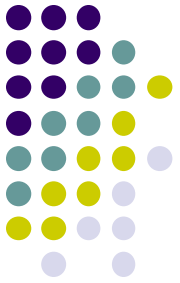
可能的剩余空间。

|        | F[0] | F[1] | F[2] | F[3] | F[4] | F[5] | F[6] | F[7] | F[8] | F[9] | F[10] |
|--------|------|------|------|------|------|------|------|------|------|------|-------|
| 未装之前：  |      |      |      |      |      |      |      |      |      |      | T     |
| 6不装/装： |      |      |      |      | T    |      |      |      |      |      | T     |
| 5不装/装： |      |      |      |      | T    | T    |      |      |      |      | T     |
| 3不装/装： |      | T    | T    |      | T    | T    |      | T    |      |      | T     |
| 8不装/装： |      | T    | T    |      | T    | T    |      | T    |      |      | T     |

剩余的最小空间：从F[0]开始第一个为真的数组元数的下标。

核心代码：





## 思考2:

背包的空间复杂度是 $n*m$

当数据太大时，存不下，该程序每层的答案只跟它的上一层相关，因此，可以使用两种方法节省空间：

1.滚动数组。

2.降成一维。

|   | 0 | 1 | 2 | 3 | 4 | $j$ |   |   |   |   | $M$ |
|---|---|---|---|---|---|-----|---|---|---|---|-----|
| 0 | 0 | 0 | 0 | 0 | 0 | 0   | 0 | 0 | 0 | 0 | 0   |
| 1 | 0 |   |   |   |   |     |   |   |   |   |     |
| 2 | 0 |   |   |   |   |     |   |   |   |   |     |
| 3 | 0 |   |   |   |   |     |   |   |   |   |     |

Diagram illustrating the 0-1 knapsack problem with a 4x11 grid. The columns represent capacity  $j$  (0 to  $M$ ) and the rows represent items  $i$  (0 to 3). The grid shows the state of the DP table. Annotations include:

- An arrow pointing to the cell at  $(i=2, j=4)$  with the text "要选第3个" (Select the 3rd item).
- An arrow pointing to the cell at  $(i=3, j=4)$  with the text "不选第3个" (Do not select the 3rd item).
- An arrow pointing to the cell at  $(i=3, j=4)$  with the text "4个空间全都给前两个物品来使用" (Use all 4 spaces for the first two items).
- A value "10" is written near the cell at  $(i=2, j=4)$ .

# 滚动数组



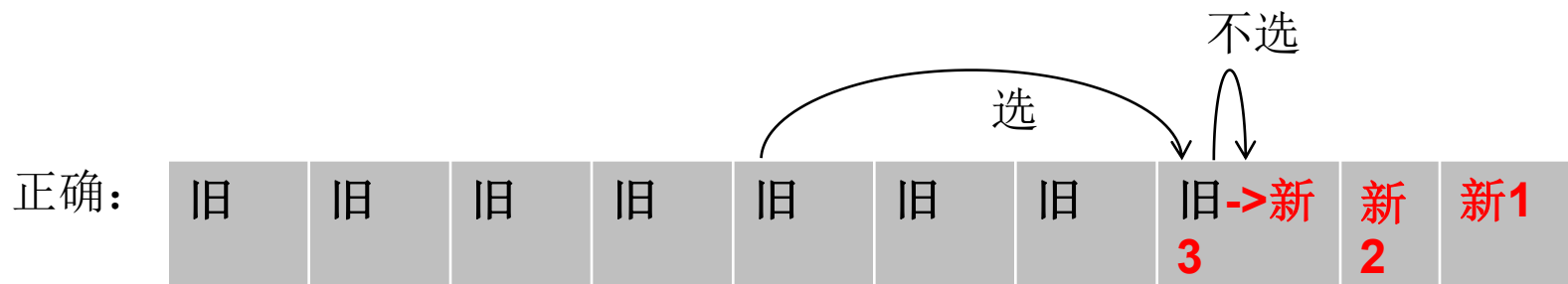
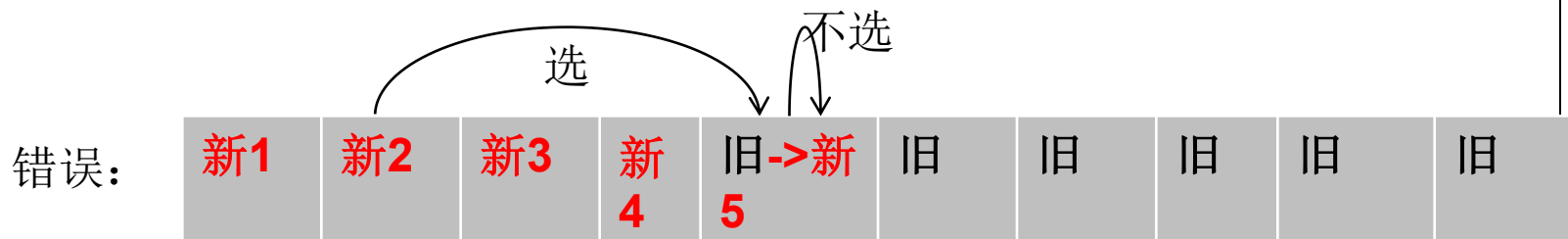
- 滚动数组主要作用是空间优化，常应用于递推的动态规划。
- 因为动规规划的无后效性，所以新的解计算出来以后与以前的再无关系，所以可以覆盖掉。
- 如01背包中， $f[i][j]$ 由 $f[i-1][j-w[i]]$ 转移得到，那么我们定义状态 $f[2][j]$ ， $f[(i-1)\%2][j]$ 表示第 $i-1$ 个物品的解， $f[i\%2][j]$ 表示第 $i$ 个物品的解，则实现了0/1滚动。

|       |   |   |   |   |
|-------|---|---|---|---|
| 1     | * | * | * | * |
| 2     | * | * | * | * |
| 3     | * | * | * | * |
| ...   | * | * | * | * |
| $i-1$ | * | * | * | * |
| $i$   | * | * | * | * |
| ...   | * | * | * | * |



|   |   |   |   |   |
|---|---|---|---|---|
| 0 | * | * | * | * |
| 1 | * | * | * | * |

# 降成一维

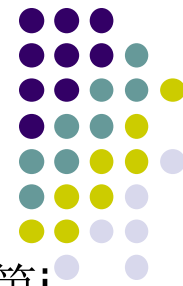


```
For (int i=1;i<=n;i++)  
    for (j=m;j>=0;j--)  
        if(j>=w[i])  
            f[j]=max(f[j],f[j-w[i]]+v [i]);
```

01背包最终代码:

```
For (int i=1;i<=n;i++)  
    for (j=m;j>=w[i];j--) //为刷新的<w[i]的部分就代表不能选的情况  
        f[j]=max(f[j],f[j-w[i]]+v [i]);
```

## 2. 完全背包问题



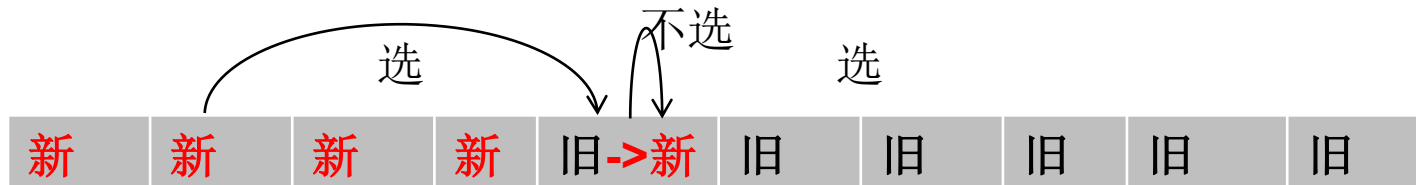
有 $N$ 种物品和一个容量为 $V$ 的背包，每种物品都有无限件可用。第 $i$ 种物品的重量是 $w[i]$ ，价值是 $v[i]$ 。求解将哪些物品装入背包可使这些物品的费用总和不超过背包容量，且价值总和最大。

一维01 背包：不受本轮刷新的影响，倒着做

```
For (int i=1;i<=n;i++)  
    for (j=m;j>=w[i];j--)  
        f[j]=max(f[j],f[j-w[i]]+v[i]);
```

完全背包：利用本轮选过后产生状态，顺着做，该物品选过后还可以选。

```
For (int i=1;i<=n;i++)  
    for (j=w[i];j<=m;j++)  
        f[j]=max(f[j],f[j-w[i]]+v [i]);
```



# 3.多重背包



- 在N种物品取出若干件放进一个空间容量为M的背包里，每件物品所占的空间为 $W_i$ ，与之相对应的价值为 $v_i$ 。每种物品最多可以选p个。求出该背包能装载的最大价值。（不考虑物品间的缝隙）

N=5 M=10 P=2 输出：24

空间  $W_i$  Vi价值

2 8  
8 6  
3 4  
5 4  
3 3

2 10

|     |   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | $j$ | $M$ |
|-----|---|---|---|---|---|---|---|---|---|---|-----|-----|
| 0   | 0 | 0 | 0 | 0 | 0 |   |   |   |   |   |     |     |
| 1   | 0 |   |   |   |   |   |   |   |   |   |     |     |
| 2   | 0 |   |   |   |   |   |   |   |   |   |     |     |
| 3   | 0 |   |   |   |   |   |   |   |   |   |     |     |
| 4   | 0 |   |   |   |   |   |   |   |   |   |     |     |
| 5   | 0 |   |   |   |   |   |   |   |   |   |     |     |
| $N$ | 0 |   |   |   |   |   |   |   |   |   |     |     |

当 $p=3$ 时，有4种选择

增加第三层循环for( $k$ .....)

写出状态转移方程或递推数组



# 第一步：首先实现例3

## 第二步：改一改：每种物品最多可以选 $p[i]$ 个



第一步：选2个  
输入：

5 10 2  
2 8  
8 6  
3 4  
5 4  
3 3

输出：24

第二步：选 $p[i]$ 个  
输入：

5 1000  
80 20 4  
40 50 9  
30 50 7  
40 30 6  
20 20 1

输出：1040

多重背包核心代码：  
**For**(int i=1;i<=n;i++)  
  **for**(int j=m;j>=**0**;j--)\物品空间随k变化无法确定  
    **for**(int k=0;k<=s[i];k++)  
      **if**(j-k\*w[i]>=0)  
        f[j]=max(f[j],f[j-k\*w[i]]+k\*v[i])

思考：多重背包降成01优化，3for降2for

1.暴力降维 80 20 13 结果时间复杂度不变

变成 80 20  
80 20  
80 20  
80 20  
.....

2.二进制优化降2for：类似于人民的组合，参考答案P357

$w[i]$   $v[i]$   $p[i]$  设 $p[i]=13$

变成：  $w[i]$   $v[i]$   
           $2w[i]$   $2v[i]$   
           $4w[i]$   $4v[i]$   
           $6w[i]$   $6v[i]$

这四种选或不选，就可以组合出0-13个选择

# 4.混合背包问题



有的物品只能选或不选，有的物品可以选无限件，有的物品可以选有限件

输入样例：

10 3

2 1 0 \\无限件,完全背包

3 3 1 \\选或不选, 01背包

4 5 4 \\有限件, 多重背包

输出样例：

11

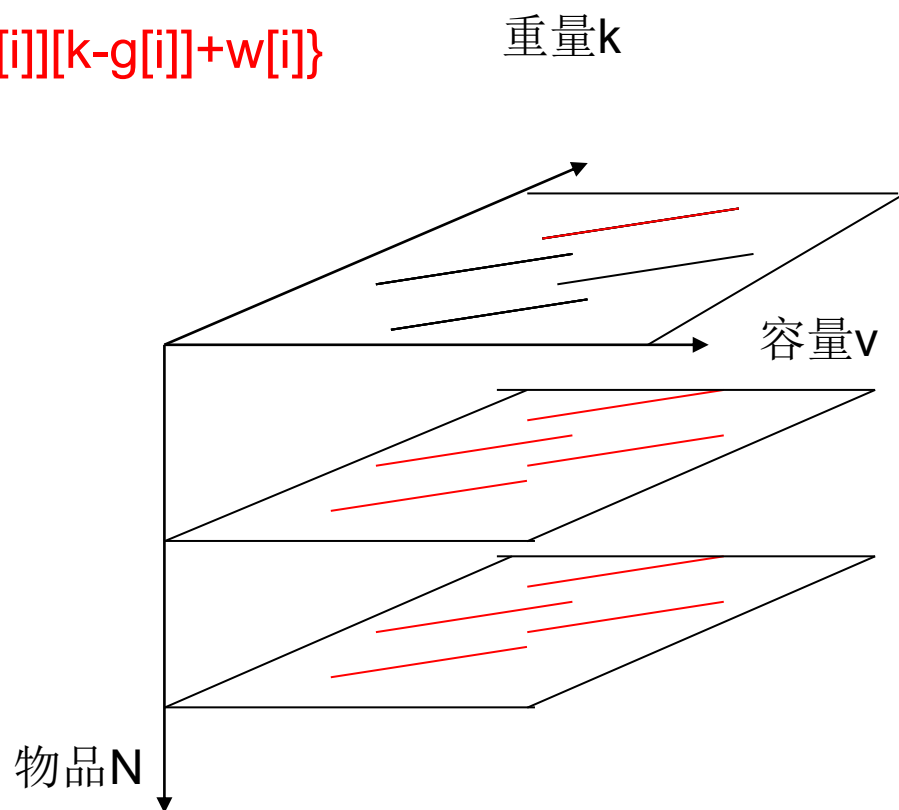
P358自学，核心代码：

```
For(int i=1;i<=n;i++)
If (p[i]==0) \\ 完全背包
    {for (int j=v[i];j<=m;j++)
        f[j]=max(f[j],f[j-w[i]]+v[i]);
    }
else \\01和多重
    {for(int j=m;j>=w[i];j- -)
        for(int k=1; k<=p[i];k++)
            f[j]=max(f[j],f[j-k*w[i]]+k*v[i])}
```

# 二维费用背包



- 二维费用背包是指有两种不同的费用，如求放N件物品到**体积V**和**承重G**的背包中的**最大价值**
- 根据01背包可得， $f[i][j][k]$ 表示前i件物品放到容量为j承重为k的背包中的最大价值，则有方程：
  - $f[i][j][k] = \max\{f[i-1][j][k], f[i-1][j-v[i]][k-g[i]]+w[i]\}$
- 同样可以优化成二维的 $f[j][k]$ ，则
  - $f[j][k] = \max\{f[j-v[i]][k-g[i]]+w[i]\}$



# 分组背包



- 分组背包是指将物品分为若干组，每组中的物品相互排斥，即每组中的物品最多只能选择一个
- 则设 $f[k][j]$ 表示前 $k$ 组物品放入体积为 $j$ 的背包中的最大价值。则有方程：
- $f[k][v] = \max\{f[k-1][v], f[k-1][v-c[i]] + w[i]\}$  物品 $i$ 属于第 $k$ 组
- 伪代码如下：
  - for 所有的组 $k$
  - for  $v=V..0$
  - for 所有的 $i$ 属于组 $k$ //写在最内层
  - $f[v] = \max\{f[v], f[v-c[i]] + w[i]\}$

# 有依赖的背包



- 是指物品之间存在依赖关系，如2006年NOIP的“金明的预算方案”
- 此种类型的题目其实可以转化为分组背包，因为选择主件及其某种附件的方法所组成的集合是互斥的，那么这个集合就是一个分组。

# 背包的方案总数



- 0/1背包
- 0/1背包降维
- 完全背包
- 多重背包
- 混合背包
- 二维费用背包
- 分组背包
- 有依赖的背包

● 方案总数

```
for(i=1;i<=n;i++)  
  for(j=1;j<=m;j++)  
  {  
    f[i][j]=f[i-1][j];  
    if (j<=w[i])  
      f[i][j]+=f[i-1][j-w[i]];  
  }
```

# 练习1：采药(medic.pas)



辰辰是个天资聪颖的孩子，他的梦想是成为世界上最伟大的医师。为此，他想拜附近最有威望的医师为师。医师为了判断他的资质，给他出了一个难题。医师把他带到一个到处都是草药的山洞里对他说：“孩子，这个山洞里有一些不同的草药，采每一株都需要一些时间，每一株也有它自身的价值。我会给你一段时间，在这段时间里，你可以采到一些草药。如果你是一个聪明的孩子，你应该可以让采到的草药的总价值最大。”

如果你是辰辰，你能完成这个任务吗？

## 输入格式 **Infut Format**

输入的第一行有两个整数 $T$  ( $1 \leq T \leq 1000$ ) 和 $X$  ( $1 \leq X \leq 100$ )，用一个空格隔开， $T$ 代表总共能够用来采药的时间， $X$ 代表山洞里的草药的数目。接下来的 $X$ 行每行包括两个在1到100之间（包括1和100）的整数，分别表示采摘某株草药的时间和这株草药的价值。

## 输出格式 **Outfut Format**

输出包括一行，这一行只包含一个整数，表示在规定的时间内，可以采到的草药的最大总价值。

## 样例输入 **Samfle Infut**

```
70 3 //注意此题第2个数是数量
71 100
69 1
1 2
```

## 样例输出 **Samfle Outfut**

3